

Improving Security and Performance of Financial RESTful APIs Using ChatGPT in ASP.NET Core

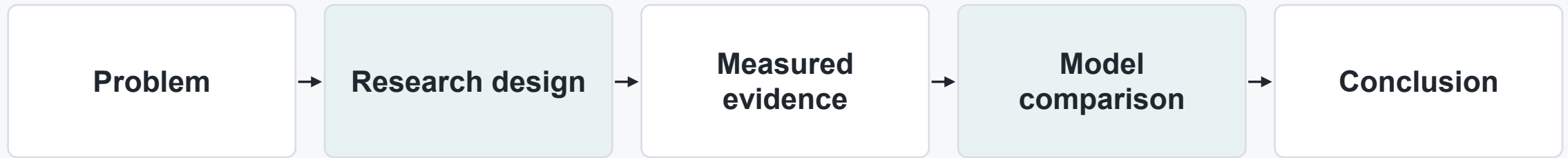
Zaw Ye Htut Ko | G6519692

Master of Science in Information Technology
Advisor: Assistant Professor Dr. Darun Kesrarat

Evidence first

Model proposes.
Evidence decides.

Defense Roadmap



Problem -> controlled method -> measured results -> model trade-offs -> defensible conclusion

One API, Three Failure Modes

Security

Headers, dependencies,
authentication, authorization

Performance

Latency, throughput, errors,
stress recovery

Maintainability

Code smells, complexity,
duplication, quality gate

Research Gap

Dominant prior focus

Code generation

Completion accuracy

Developer productivity



This thesis

Tool findings as prompts

Branch-isolated fixes

Independent remeasurement

Research Questions

RQ1

Can tool-guided ChatGPT recommendations reduce measured code-quality findings without regression?

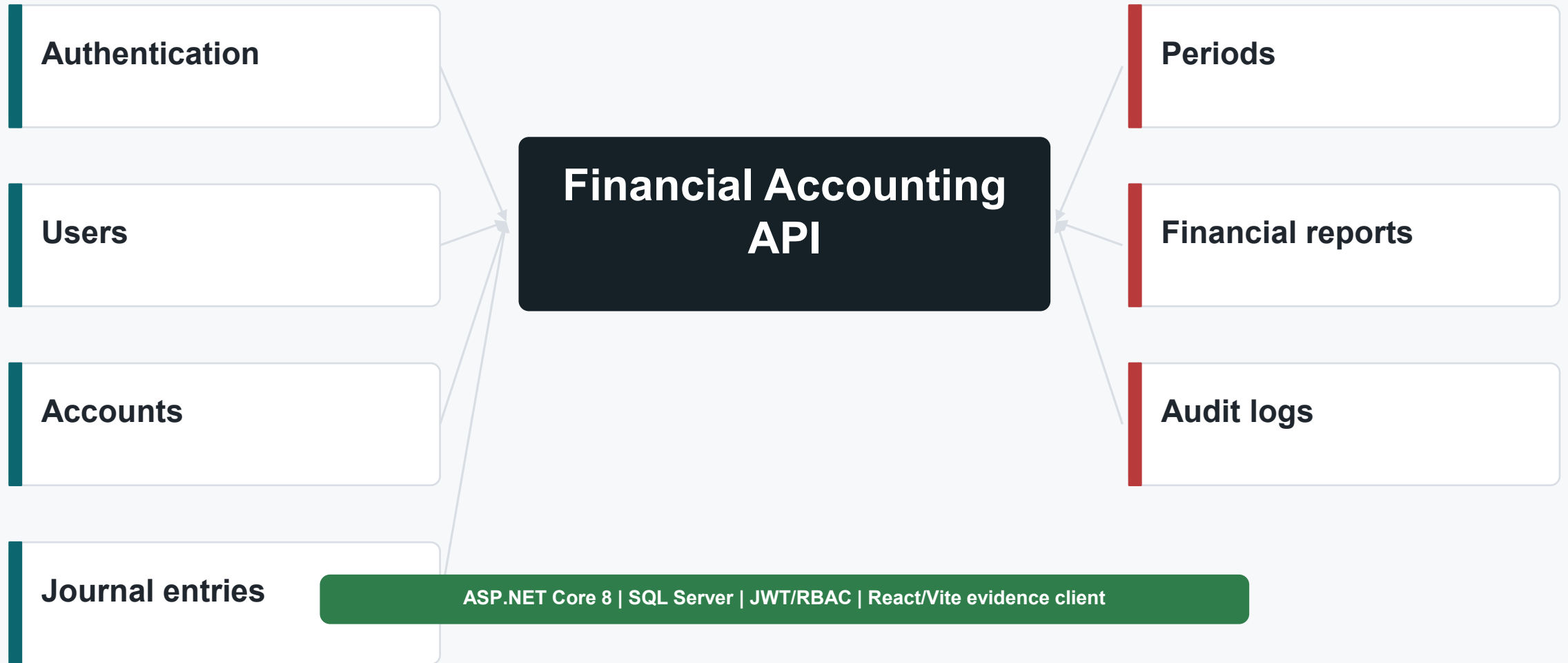
RQ2

Can they clear gated security alerts while preserving role-based access?

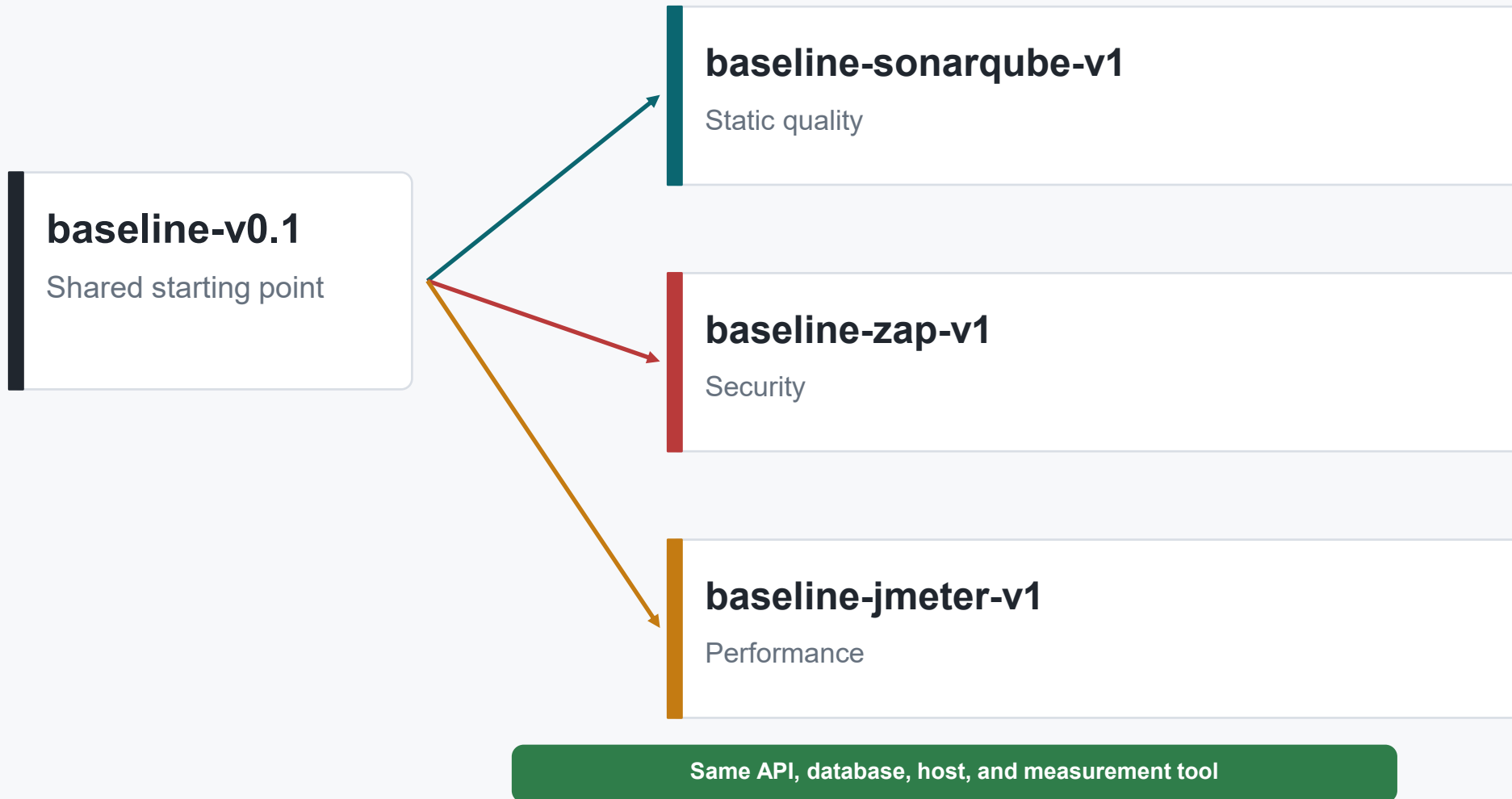
RQ3

Can they reduce tail latency and maintain zero-error behavior under load?

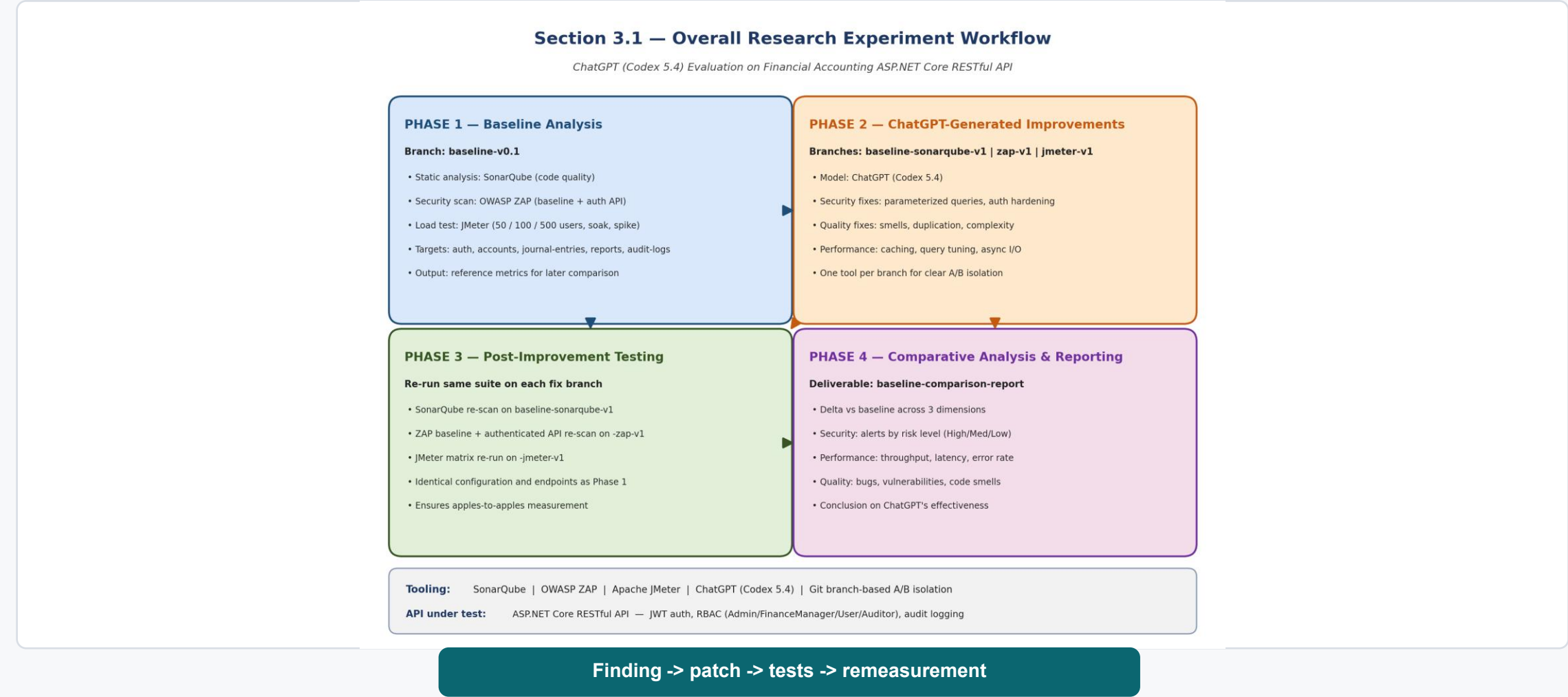
System Under Study



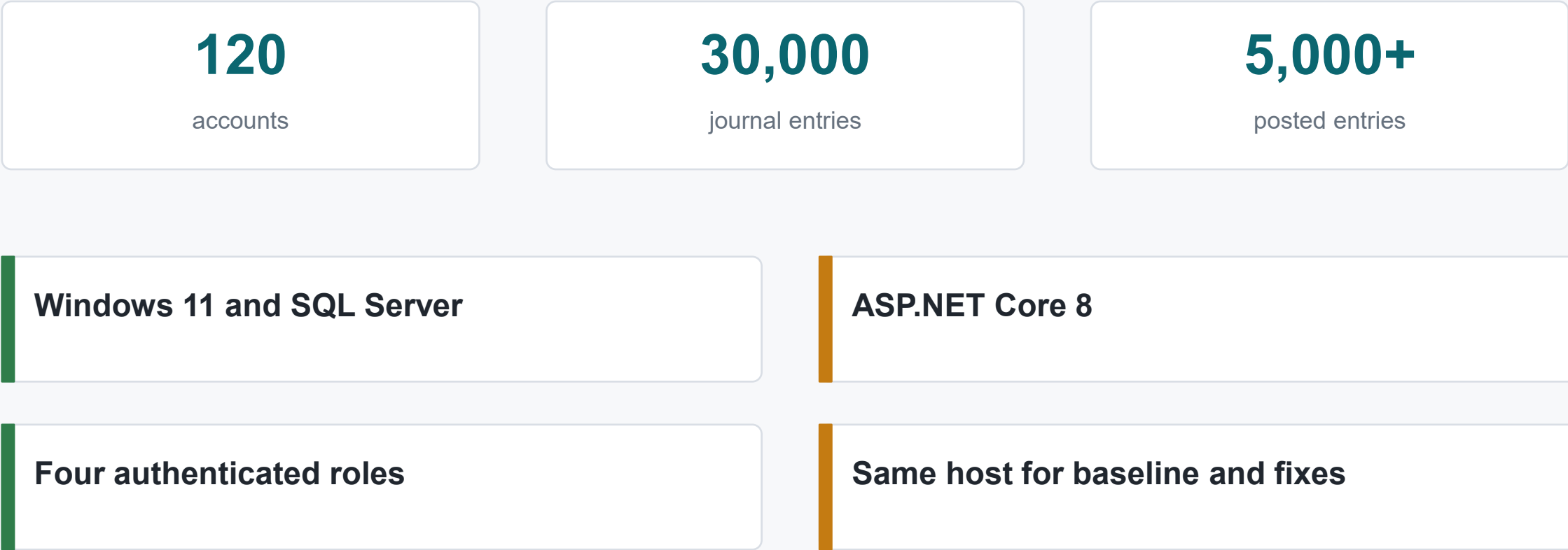
Branch-Isolated Research Design



Four-Phase Evidence Workflow



Environment and Seeded Dataset



Evaluation Gates

SonarQube

Static quality

OWASP ZAP

Security alerts

Apache JMeter

Load behavior

dotnet test

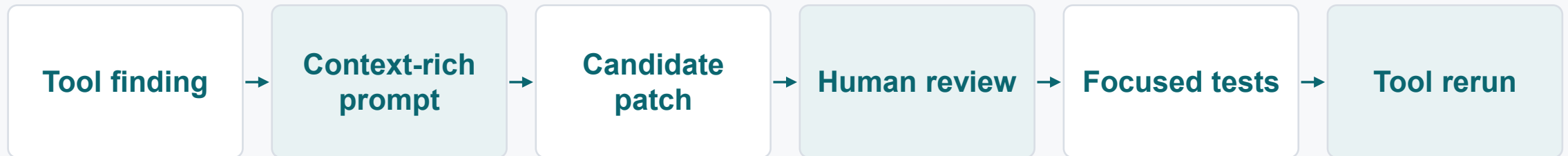
Regression
safety

Git evidence

Traceability

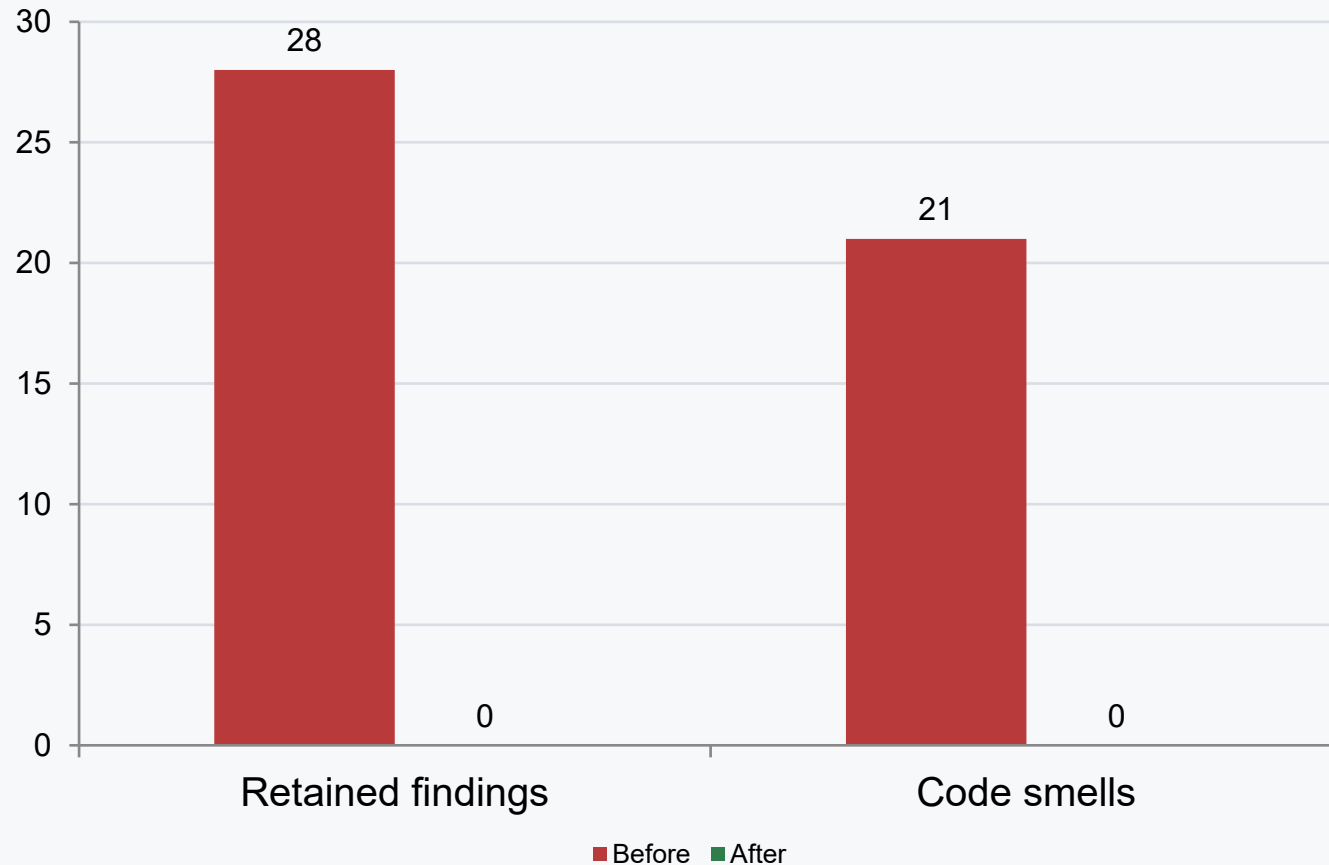
No single metric is overall quality

Human-in-the-Loop Use of ChatGPT/Codex



Acceptance is determined by evidence, not model confidence

SonarQube: Clean Closure Without Coverage Loss



Quality Gate: OK

Coverage: 74.3% -> 74.3%

Duplicated lines: 0.0% -> 0.0%

OWASP ZAP: Business-Endpoint Gates Cleared

Baseline scan

Medium 2 -> 0

Low 6 -> 0

Informational 3 -> 4

Authenticated API scan

Low 3 -> 0

Informational 5 -> 5

High 0 | Medium 0 | Low 0 after
remediation

JMeter: Five Workload Profiles

p50

Normal load

p100

Higher concurrency

p500

Heavy concurrency

Soak

Sustained behavior

Spike

Burst and recovery

p95 and p99 latency



Throughput

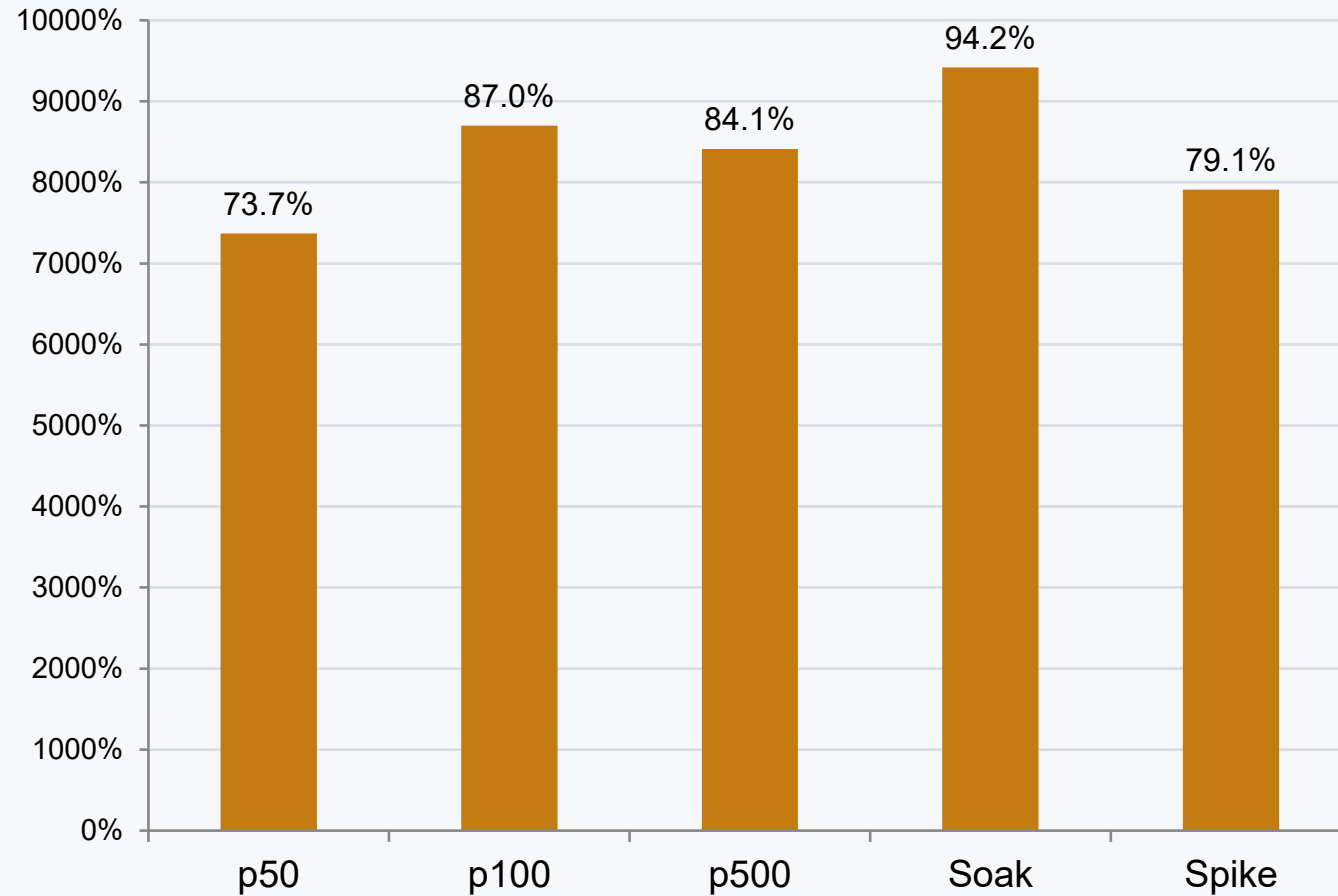


Error rate



Drift

JMeter: Large p95 Reductions, One Honest Caveat



Core reliability

0.00% error rate on p50, p100, and p500

Caveat retained

Mixed-workload soak/spike drift heuristics still fail

Integrity Was a Non-Negotiable Constraint

CHECK 01

JWT authentication

CHECK 02

Role-based authorization

CHECK 03

Balanced debit and credit

CHECK 04

Accounting-period rules

CHECK 05

Idempotency and concurrency

CHECK 06

Audit writes

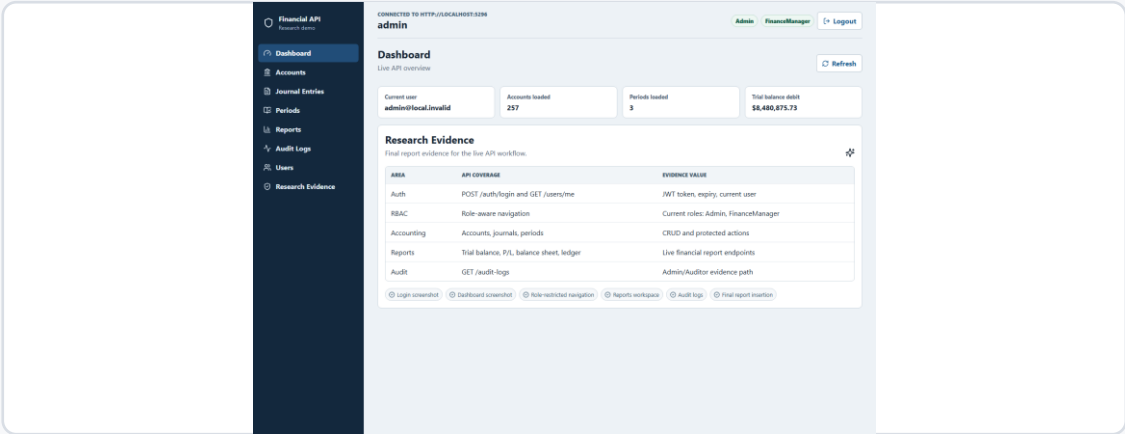
CHECK 07

Reconciliation integrity

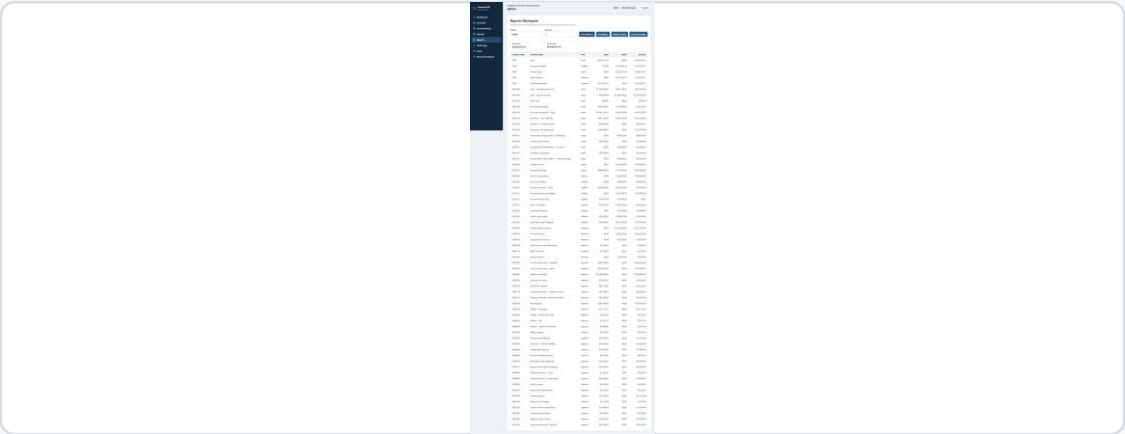
CHECK 08

Regression tests

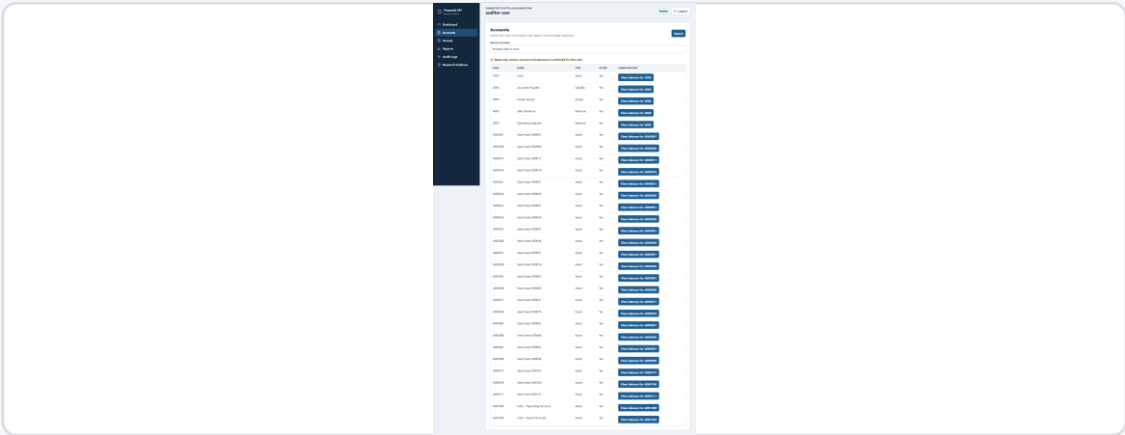
Working Application Evidence



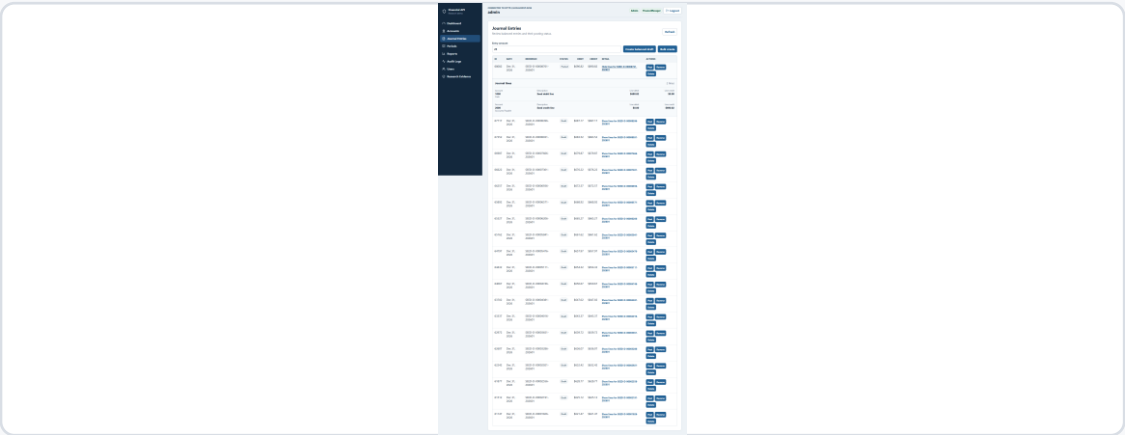
Admin dashboard and live API counts



Trial balance from reports API



Auditor read-only navigation



Debit and credit line evidence

GPT-5.4 vs GPT-5.5: Trade-offs, Not a Universal Winner

Area	GPT-5.4	GPT-5.5	Reading
SonarQube	0 code smells; 74.3% coverage	0 code smells; 74.3% coverage	Tie
ZAP	Cleaner residual output	Business gate passes; more residuals	GPT-5.4 cleaner
JMeter	Selected tail wins; p50 regression	Balanced profiles; best spike tail	GPT-5.5 balanced
Patch size	Larger	Smaller	GPT-5.5 lower change risk

New-API v2 Controlled Retest

Measure	GPT-5.4	GPT-5.5
Sonar issues	0	0
Coverage	76.2%	74.8%
ZAP verified alerts	12	12
Public tests	152 passed	151 passed
JMeter	No valid candidate	Frozen candidate

Contributions

Empirical

Measured before/after
evidence on one production-
complexity API

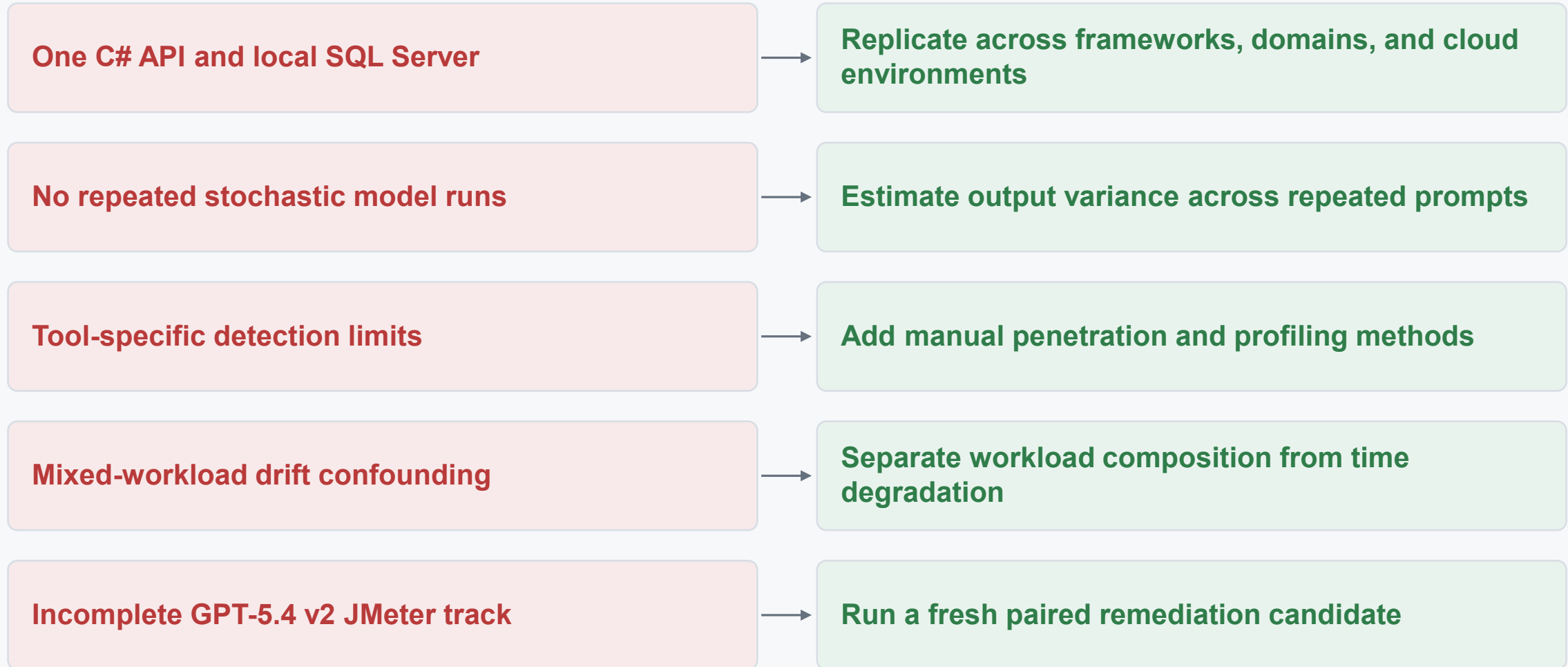
Methodological

Branch-isolated, scanner-
gated acceptance workflow

Practical

Traceable prompts, patches,
tests, screenshots, and tool
outputs

Limitations and Future Work



LIMITATION

NEXT EVIDENCE

Conclusion

1

Measured gains across quality, security, and latency

2

Human review and independent tools remained essential

3

The strongest contribution is the controlled audit-to-fix workflow

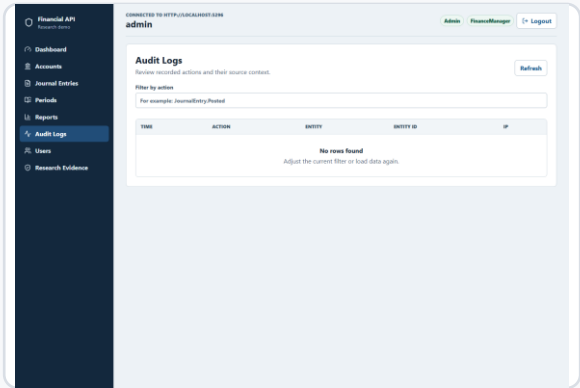
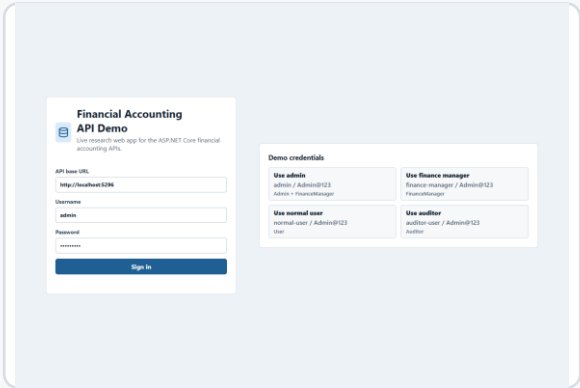
Use the model to propose. Use evidence to decide.

Branch and Commit Provenance

Track	Branch	Evidence
Original baseline	baseline-v0.1	Shared starting point
Original Sonar	baseline-sonarqube-v1	Static-quality rerun
Original ZAP	baseline-zap-v1	Baseline and authenticated scans
Original JMeter	baseline-jmeter-v1	Five workload profiles
GPT-5.5 comparison	codex/gpt-5.5-comparison	Paired model evidence

Endpoint and Role Coverage

Endpoint group	Admin	FinanceManager	User	Auditor
/auth	Full	Role-based	Own scope	Limited
/users	Full	Role-based	Own scope	Limited
/accounts	Full	Role-based	Own scope	Read
/journal-entries	Full	Role-based	Own scope	Read
/periods	Full	Role-based	Own scope	Read
/reports	Full	Role-based	Own scope	Read
/audit-logs	Admin	Read	Own scope	Read



Full SonarQube Metrics

Metric	Baseline	After
Retained findings	28	0
Code smells	21	0
Coverage	74.3%	74.3%
Duplicated lines	0.0%	0.0%
Quality Gate	OK	OK

Full OWASP ZAP Outcomes

Scan	Severity	Before	After
Baseline	Medium	2	0
Baseline	Low	6	0
Baseline	Informational	3	4
Authenticated	Low	3	0
Authenticated	Informational	5	5

Full JMeter Total p95 Results

Profile	Baseline ms	After ms	Reduction
p50	49.45	13.00	73.7%
p100	261.95	34.00	87.0%
p500	176.00	28.00	84.1%
Soak	970.95	56.00	94.2%
Spike	6795.00	1420.00	79.1%

Patch Size and Regression Evidence

Track	GPT-5.4 files	GPT-5.5 files	Verification
SonarQube	13	13	0 smells; gate OK
ZAP	7	4	Business gate passed
JMeter	13	8	0% errors; drift gate passed in normalized runs

New-API v2 Detailed Results

Measure	GPT-5.4	GPT-5.5
Sonar issues	0	0
Coverage	76.2%	74.8%
Cognitive complexity	340	329
ZAP verified alerts	12	12
Public tests	152 passed	151 passed
JMeter	No valid candidate	Frozen candidate

Threats to Validity

Internal

Branch differences

Tool configuration

Cache and warm-up effects

Construct

Scanner coverage

Metric definitions

Drift heuristic

External

One API

One host

One language and domain

Conclusion

No repeated model runs

Incomplete v2 JMeter pair

Prompt and Human-Effort Traceability



- No suppression
- No weakened test
- No feature deletion
- No unverified acceptance

The screenshot shows a web application interface for 'Financial API Research Evidence'. The left sidebar contains a navigation menu with items: Dashboard, Accounts, Journal Entries, Periods, Reports, Audit Logs, Users, and Research Evidence (which is highlighted). The main content area is titled 'Research Evidence' and displays a table of evidence. The table has three columns: AREA, API COVERAGE, and EVIDENCE VALUE. The data rows are as follows:

AREA	API COVERAGE	EVIDENCE VALUE
Auth	POST /auth/login and GET /users/me	JWT token, expiry, current user
RBAC	Role-aware navigation	Current roles: Admin, FinanceManager
Accounting	Accounts, journals, periods	CRUD and protected actions
Reports	Trial balance, P/L, balance sheet, ledger	Live financial report endpoints
Audit	GET /audit-logs	Admin/Auditor evidence path

At the bottom of the table, there are several links: Login screenshot, Dashboard screenshot, Role-restricted navigation, Reports workspace, Audit logs, and Final report creation.

Selected References

- Hou et al. (2024) - LLMs for software engineering review
- Khoury et al. (2023) - Security of ChatGPT-generated code
- Sandoval et al. (2023) - Security implications of LLM assistants
- Lenarduzzi et al. (2020) - Limits of SonarQube rules
- Zhang et al. (2023) - Web API issues and expectations